

TTK4190 - GUIDANCE, NAVIGATION AND CONTROL OF MARINE
CRAFT, AIRCRAFT AND DRONES

Assignment 2: Part 1 Marine Craft Modeling

Authors:

Audun Bagøyen Hustad
Bendik Letting Skari
Fredrik Siem Taklo
William Hammer

7th October 2025

Table of Contents

1	Rigid-Body Kinetics of a Rectangular Prism	1
1a	Yaw moment of inertia at center of gravity	1
1b	Yaw moment of inertia at coordinate origin	1
1c	Mass and Coriolis matrix	1
1d	Skew-symmetric Coriolis matrix	2
1e	Current invariance of Coriolis matrix	2
2	Hydrostatics of a Rectangular Prism - Bendik	4
2a	Volume displacement	4
2b	Water plane area	4
2c	Period in heave	4
2d	Analytical expression	4
2e	Stability	4
3	Added Mass and Coriolis	5
3a	Simplifying added mass matrix	5
3b	Added Coriolis matrix	5
4	Implementing the Maneuvering Model in Matlab	6
4a	Added mass and Coriolis matrices	6
4b	Added damping	7
4c	Step responses	9
4c.1	First step response	9
4c.2	Second step response	10
4d	Vessel dynamics	12
	Bibliography	13
	Appendix	14
A	Code for task 4a	14
B	Code for task 4b	15
C	Code for task 4c	17

1 Rigid-Body Kinetics of a Rectangular Prism

1a Yaw moment of inertia at center of gravity

For a rectangular block with homogeneous mass (like our simplified ship model) the yaw moment of inertia about the center of gravity I_z^{CG} can be written as

$$\begin{aligned} I_z^{CG} &= \frac{1}{12}m(L^2 + B^2) \\ &= \frac{1}{12}17.0677 \cdot 10^6(161^2 + 21.8^2) \\ &\approx 3.7544 \cdot 10^{10} \end{aligned} \tag{1}$$

where L is the ship's length along its x-axis and B is its width along its y-axis.

The two products of inertia I_{xy}^{CG} and I_{yx}^{CG} become zero because the body is symmetric and homogeneous. For every mass element on one axis, there is an equal mass element on the other.

1b Yaw moment of inertia at coordinate origin

The distance from the coordinate origin CO to the center of gravity CG is given as

$$\mathbf{r}_{bg}^b = \begin{bmatrix} x_g \\ y_g \\ z_g \end{bmatrix} = \begin{bmatrix} -3.7 \\ 0 \\ H/2 \end{bmatrix}.$$

For the moment of inertia about the z -axis, only x_g and y_g contribute:

$$d^2 = x_g^2 + y_g^2 = (-3.7)^2 + 0^2 = 13.69.$$

By the *parallel axis theorem* we obtain

$$I_z^{CO} = I_z^{CG} + md^2,$$

where $I_z^{CG} = 3.7544 \times 10^{10}$ from Problem 1a and $m = 1.70677 \times 10^7$ kg.

$$I_z^{CO} = 3.7544 \times 10^{10} + (1.70677 \times 10^7)(13.69),$$

$$I_z^{CO} \approx 3.7777 \times 10^{10} \text{ kg}\cdot\text{m}^2.$$

The ratio between the prism's and the ship's moment of inertia is then

$$\frac{I_{z, prism}^{CO}}{I_{z, ship}^{CO}} = \frac{3.7777 \times 10^{10}}{2.1732 \times 10^{10}} \approx 1.7383.$$

1c Mass and Coriolis matrix

The analytical expressions for $\mathbf{M}_{RB}$ and $\mathbf{C}_{RB}$ is defined as follows in Fossen 2021. [1]

$$\begin{aligned} \mathbf{M}_{RB} &:= \mathbf{H}^\top(\mathbf{r}_{bg}^b) \mathbf{M}_{RB}^{CG} \mathbf{H}(\mathbf{r}_{bg}^b) \\ \mathbf{C}_{RB} &:= \mathbf{H}^\top(\mathbf{r}_{bg}^b) \mathbf{C}_{RB}^{CG} \mathbf{H}(\mathbf{r}_{bg}^b) \end{aligned} \tag{2}$$

Then expand the expressions to the full matrix form.

$$\mathbf{M}_{RB}^{CG} = \begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & I_z^{CG} \end{bmatrix}, \quad \mathbf{M}_{RB} = \begin{bmatrix} m & 0 & -my_g \\ 0 & m & mx_g \\ -my_g & mx_g & I_z^{CG} + m(x_g^2 + y_g^2) \end{bmatrix}.$$

With $y_g = 0$ this reduces to

$$\mathbf{M}_{RB} = \begin{bmatrix} m & 0 & 0 \\ 0 & m & mx_g \\ 0 & mx_g & I_z^{CO} \end{bmatrix}, \quad I_z^{CO} = I_z^{CG} + m(x_g^2 + y_g^2).$$

Using Fossen (2021, Eq. 3.30) [1] specialized to (u, v, r) about CO ,

$$\mathbf{C}_{RB}(\nu) = \begin{bmatrix} 0 & 0 & -m(v + x_g r) \\ 0 & 0 & mu \\ m(v + x_g r) & -mu & 0 \end{bmatrix}.$$

1d Skew-symmetric Coriolis matrix

To verify if $\mathbf{C}_{RB}$ is skew-symmetric, is by checking if the transpose of the matrix is equal the negative of the original matrix.

$$\mathbf{C}_{RB}(\nu) = \begin{bmatrix} 0 & 0 & -m(v + x_g r) \\ 0 & 0 & mu \\ m(v + x_g r) & -mu & 0 \end{bmatrix}.$$

Its transpose is

$$\mathbf{C}_{RB}^\top(\nu) = \begin{bmatrix} 0 & 0 & m(v + x_g r) \\ 0 & 0 & -mu \\ -m(v + x_g r) & mu & 0 \end{bmatrix} = -\mathbf{C}_{RB}(\nu).$$

Hence,

$$\mathbf{C}_{RB} = -\mathbf{C}_{RB}^\top,$$

which shows that $\mathbf{C}_{RB}$ is skew-symmetric. This is desirable because the skew-symmetry implies that

$$\nu^\top \mathbf{C}_{RB}(\nu) \nu = 0,$$

for any velocity vector ν . Thus, the Coriolis and centripetal forces do no work on the system and cannot create or dissipate kinetic energy. This ensures energy conservation of the rigid-body dynamics, leading to a physically consistent model and improved numerical stability in simulations.

1e Current invariance of Coriolis matrix

The Coriolis matrix $\mathbf{C}_{RB}$ has the important property that

$$\mathbf{C}_{RB}(\nu_c) \nu_c = 0,$$

for any constant velocity vector ν_c .

This implies that if the vessel is subject to an irrotational (uniform) ocean current, the Coriolis and centripetal forces associated with the current velocity vanish. Consequently, only the *relative velocity*

$$\nu_r = \nu - \nu_c$$

appears in the equations of motion. And we have

$$\mathbf{M}_{RB}\dot{\nu} + \mathbf{C}_{RB}(\nu)\nu \equiv \mathbf{M}_{RB}\dot{\nu}_r + \mathbf{C}_{RB}(\nu_r)\nu_r$$

This ensures that the rigid-body dynamics is invariant to a constant ocean current and prevents artificial forces or moments from arising due to the choice of reference frame. It also guarantees a physically consistent model when currents are included in the simulation.

2 Hydrostatics of a Rectangular Prism - Bendik

2a Volume displacement

The volume displacement is calculated as:

$$\nabla = T[m] * B[m] * L[m] = 4.74 \times 21.8 \times 161 = 16636.452[m^3] \quad (3)$$

2b Water plane area

The water plane area is defined by the area in the xy-plane that is covered by the vessel at the surface.

$$A_{wp} = B[m] \times L[m] = 21.8 \times 161 = 3509.8[m^2] \quad (4)$$

The hydrostatic restoring force Z_{hs} can be found from:

$$Z_{hs} = A_{wp}[m^3] \times \rho[kg/m^3] \times g[m/s^2] = 3509.8 \times 1025 \times 9.81 = 35291916.45[N/m]. \quad (5)$$

2c Period in heave

$$T_3 = 2\pi \sqrt{\frac{2T}{g}} = 2\pi \times \sqrt{\frac{2 \times 4.74}{9.81}} = 6.1766[s] \quad (6)$$

2d Analytical expression

$$BG = z_g - z_b = \frac{15.8}{2} - \frac{4.74}{2} = 5.53[m] \quad (7)$$

$$GM_T = \frac{I_T}{\nabla} + z_g - z_b = \frac{L * B^3}{12 * \nabla} - BG = \frac{161 * 21.8^3}{12 * 16636.452} - 5.53 = 2.8251[m] \quad (8)$$

$$GM_L = \frac{I_L}{\nabla} + z_g - z_b = \frac{L^3 * B}{12 * \nabla} - BG = \frac{161^3 * 21.8}{12 * 16636.452} - 5.53 = 450.1838[m] \quad (9)$$

2e Stability

With a $GM_T = 2.8251$ m and $GM_L = 450.1838$ the vessel is definitely stable. If any of the values are negative, the vessel will flip. However, there are also recommended minimum values based on the type of vessel/vessel. A typical minimum value for GM_T is 100m, which we are well within. Similarly for GM_L should be bigger then 0.5 m, which we also are within by a good margin.

3 Added Mass and Coriolis

3a Simplifying added mass matrix

The task consists of converting a 6x6 M_A matrix to a 3x3 M_A matrix. The 6x6 matrix is defined by:

$$M_A = - \begin{bmatrix} X_{\dot{u}} & X_{\dot{v}} & X_{\dot{w}} & X_{\dot{p}} & X_{\dot{q}} & X_{\dot{r}} \\ Y_{\dot{u}} & Y_{\dot{v}} & Y_{\dot{w}} & Y_{\dot{p}} & Y_{\dot{q}} & Y_{\dot{r}} \\ Z_{\dot{u}} & Z_{\dot{v}} & Z_{\dot{w}} & Z_{\dot{p}} & Z_{\dot{q}} & Z_{\dot{r}} \\ K_{\dot{u}} & K_{\dot{v}} & K_{\dot{w}} & K_{\dot{p}} & K_{\dot{q}} & K_{\dot{r}} \\ M_{\dot{u}} & M_{\dot{v}} & M_{\dot{w}} & M_{\dot{p}} & M_{\dot{q}} & M_{\dot{r}} \\ N_{\dot{u}} & N_{\dot{v}} & N_{\dot{w}} & N_{\dot{p}} & N_{\dot{q}} & N_{\dot{r}} \end{bmatrix}$$

The matrix can be reduced by removing the couplings between surge, heave-roll-pitch and sway-yaw. This is done by setting all the elements to zero with these couplings. For example, setting surge-heave to zero means setting $X_{\dot{w}}$ and $Z_{\dot{u}}$ to zero. This leaves us with this matrix:

$$M_A = - \begin{bmatrix} X_{\dot{u}} & 0 & 0 & 0 & 0 & 0 \\ 0 & Y_{\dot{v}} & 0 & 0 & 0 & Y_{\dot{r}} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & N_{\dot{v}} & 0 & 0 & 0 & N_{\dot{r}} \end{bmatrix}$$

By removing zero rows and columns one gets:

$$M_A = - \begin{bmatrix} X_{\dot{u}} & 0 & 0 \\ 0 & Y_{\dot{v}} & Y_{\dot{r}} \\ 0 & N_{\dot{v}} & N_{\dot{r}} \end{bmatrix}$$

3b Added Coriolis matrix

Theorem 3.2 in Fossen 2021[1] states that the Coriolis/centripetal matrix can be derived from the system inertia matrix. Example 6.1 shows how this is done with added mass for surface vessels, directly mirroring this problem:

$$M_A = M_A^T = - \begin{bmatrix} X_{\dot{u}} & 0 & 0 \\ 0 & Y_{\dot{v}} & Y_{\dot{r}} \\ 0 & N_{\dot{v}} & N_{\dot{r}} \end{bmatrix}$$

the corresponding Coriolis/centripetal matrix can be parametrized such that

$$C_A(v_r) = -C(v_r)^T = \begin{bmatrix} 0 & 0 & Y_{\dot{v}}v_r + Y_{\dot{r}} \\ 0 & 0 & -X_{\dot{u}}u_r \\ -Y_{\dot{v}}v_r - Y_{\dot{r}}r & X_{\dot{u}}u_r & 0 \end{bmatrix} \quad (N_{\dot{v}} = Y_{\dot{r}})$$

where $v_r = [u_r, v_r, r]^T$. Heave, pitch and roll are neglected under the assumption that they are small for surface vessels.

4 Implementing the Maneuvering Model in Matlab

4a Added mass and Coriolis matrices

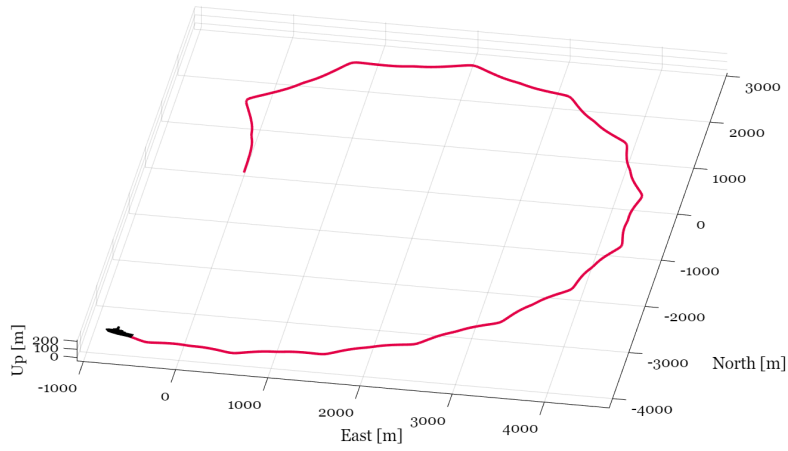


Figure 1: 4a Result path

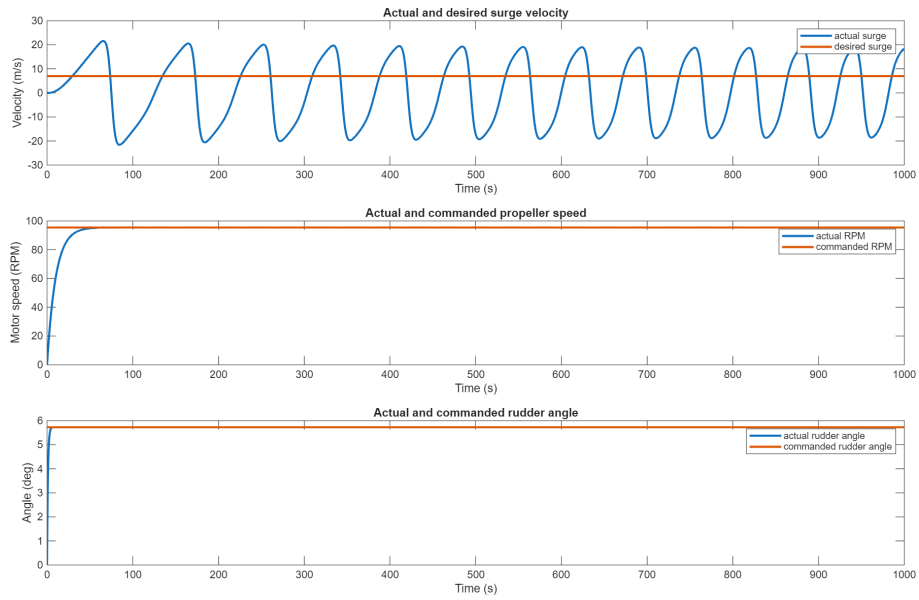


Figure 2: 4a Resulting surge velocity, propeller speed and rudder angle

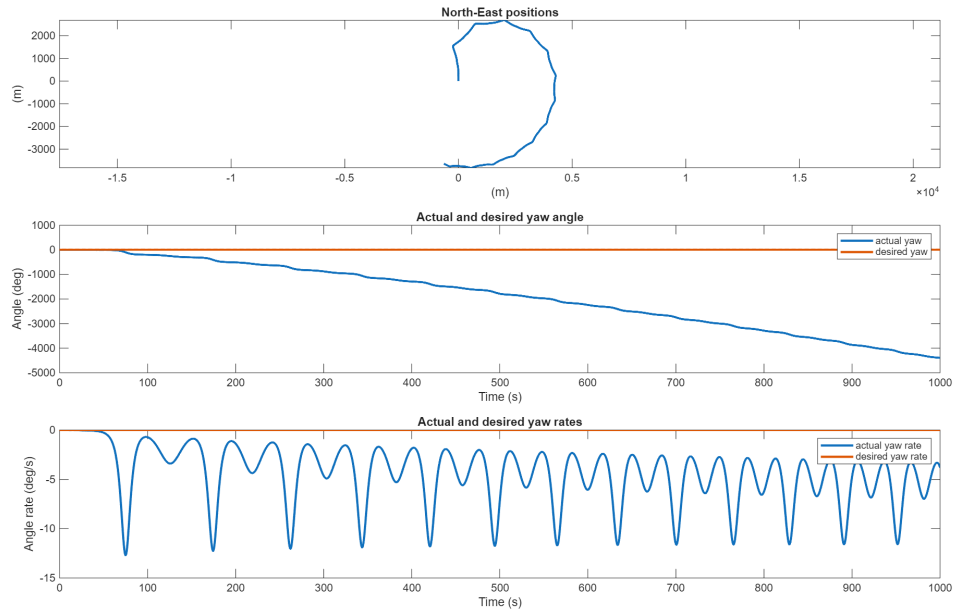


Figure 3: 4a Resulting NE path, yaw angle, yaw rate

4b Added damping

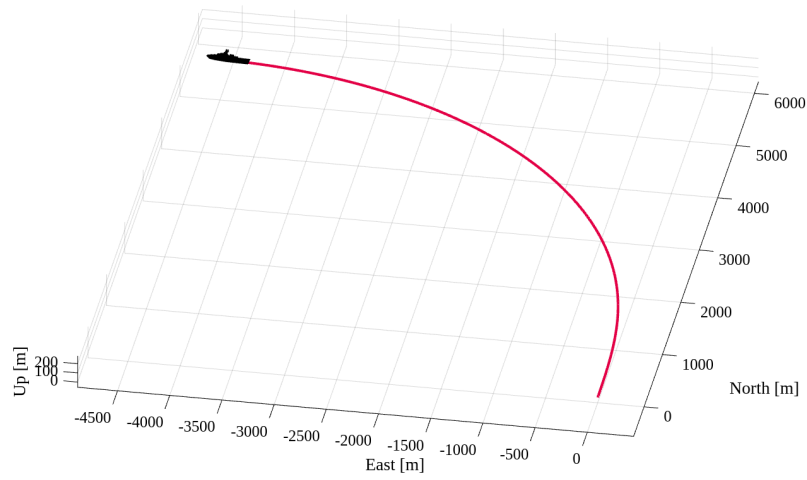


Figure 4: 4b Result path.

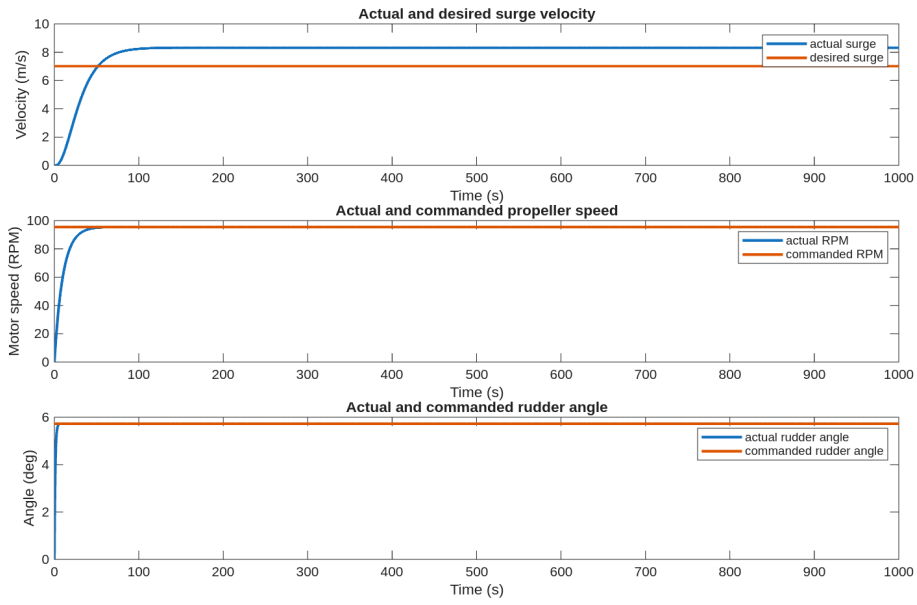


Figure 5: 4b Resulting surge velocity, propelled speed and rudder angle

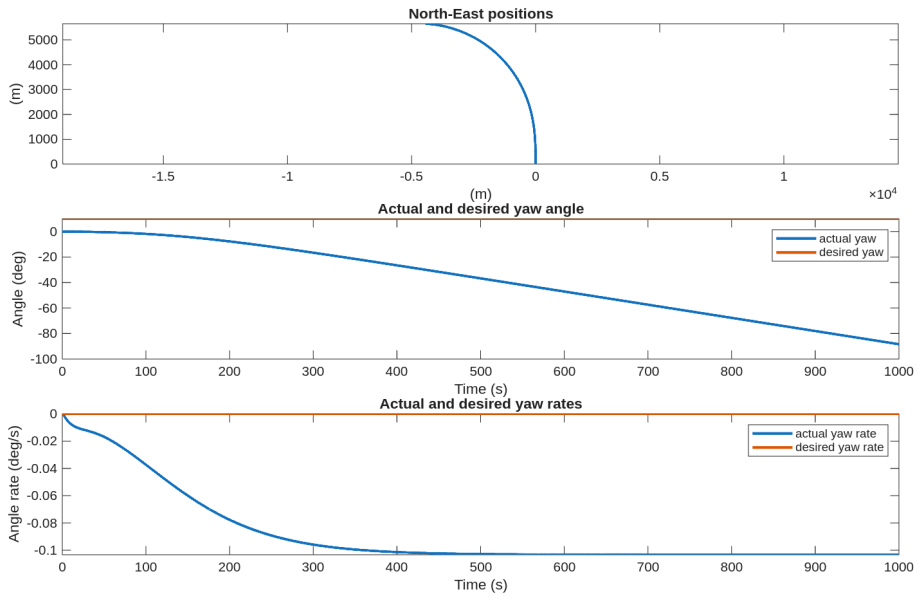


Figure 6: 4b Resulting NE path, yaw angle, yaw rate

4c Step responses

4c.1 First step response

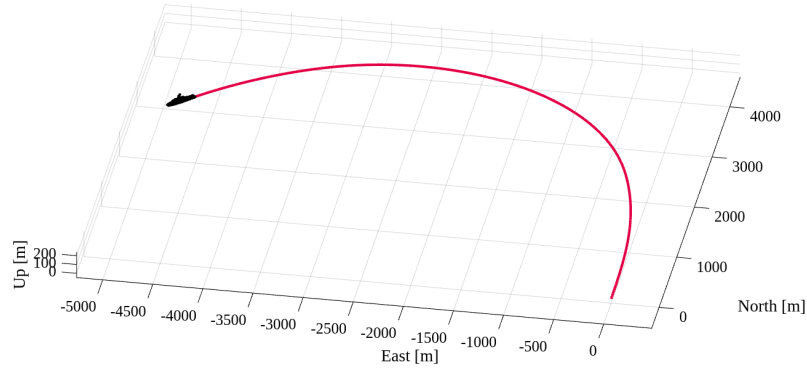


Figure 7: 4c Step response 1 (rudder): Resulting path.

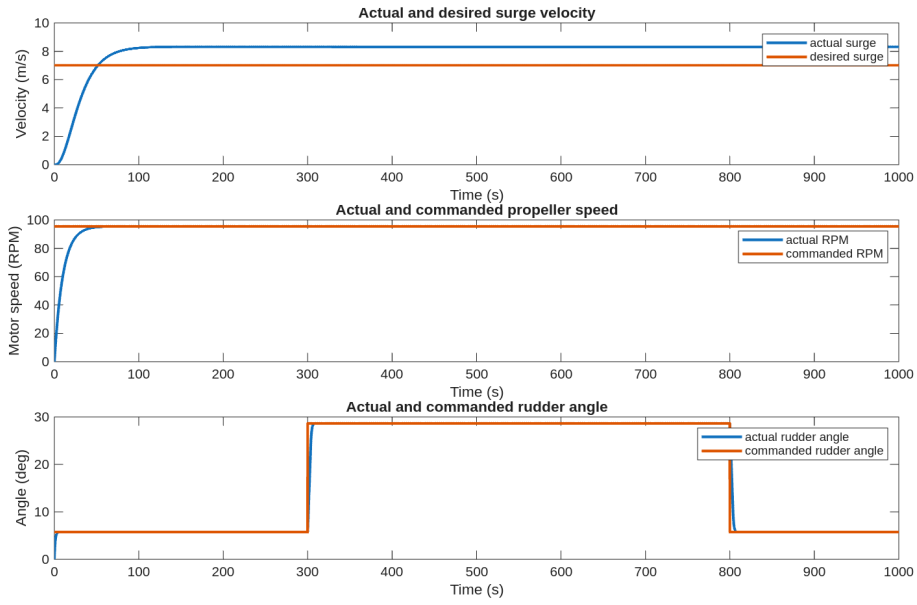


Figure 8: 4c Step response 1 (rudder): Surge velocity, propeller speed and rudder angle.

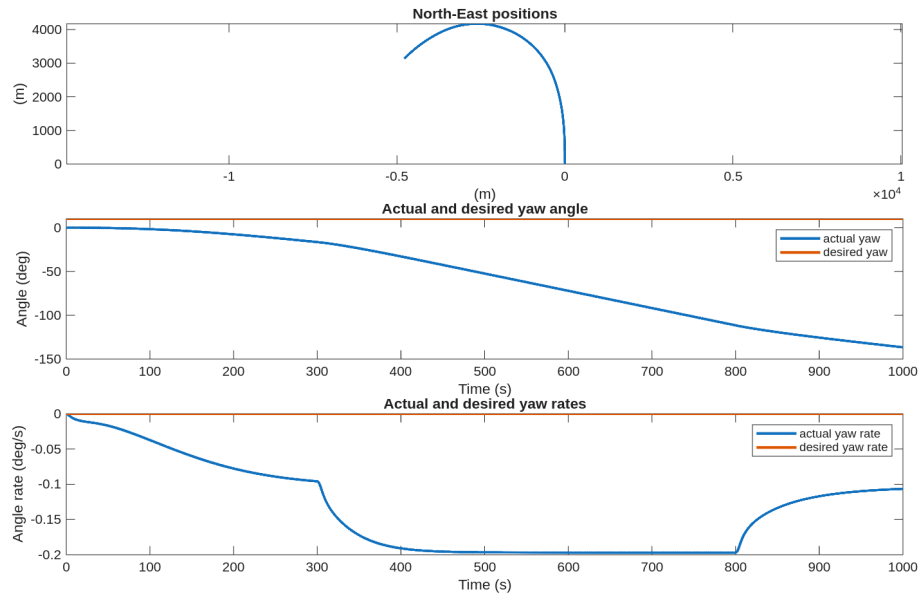


Figure 9: 4c Step response 1 (rudder): North-East path, yaw angle and yaw rate.

4c.2 Second step response

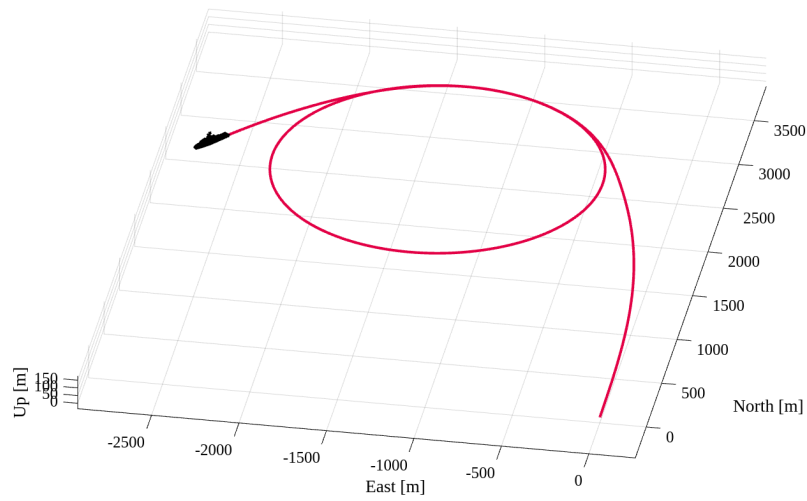


Figure 10: 4c Step response 2 (propeller): Resulting path.

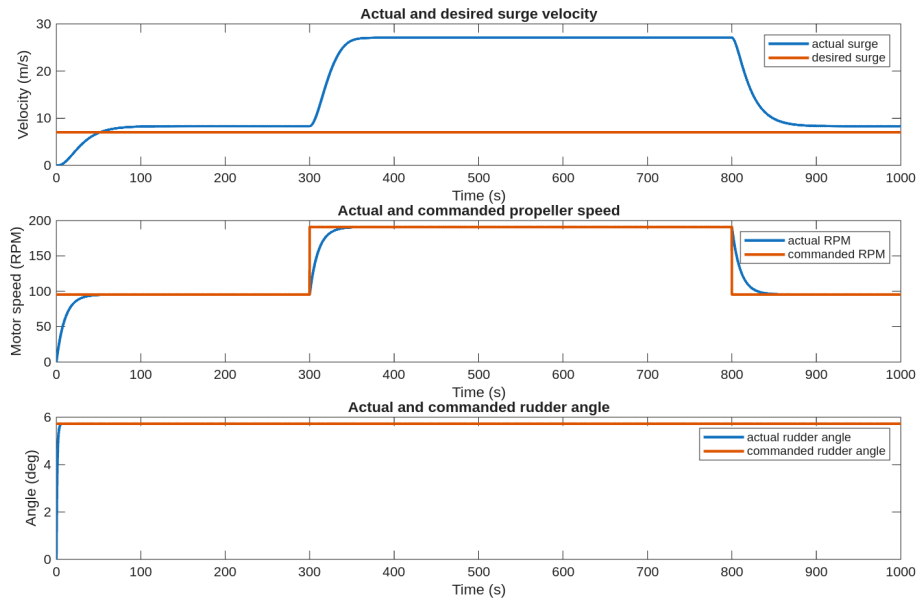


Figure 11: 4c Step response 2 (propeller): Surge velocity, propeller speed and rudder angle.

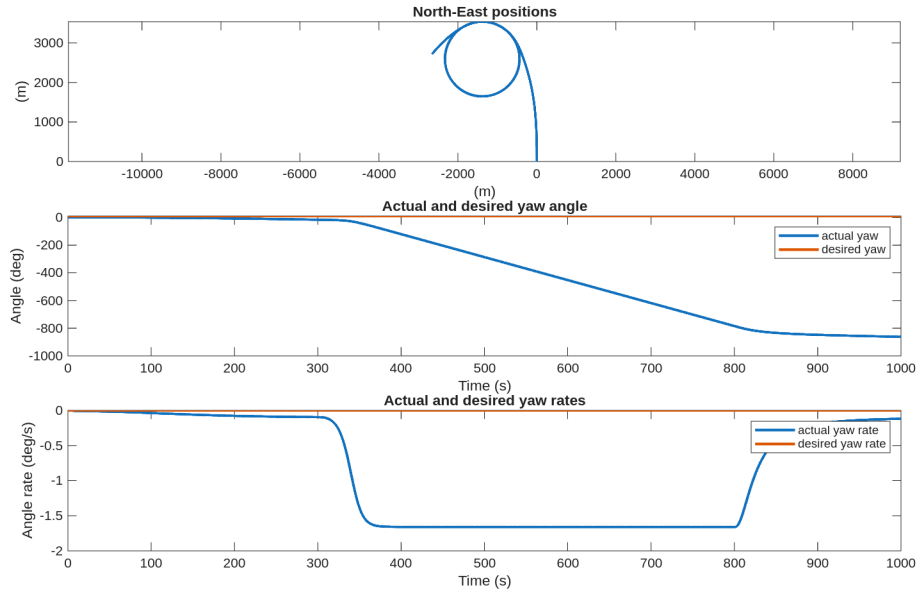


Figure 12: 4c Step response 2 (propeller): North-East path, yaw angle and yaw rate.

4d Vessel dynamics

Provided the rigid-body equations of motion for a vessel:

$$\mathbf{M}_{RB}\dot{\nu} + \mathbf{C}_{RB}(\nu)\nu = \tau_{RB} \quad (\text{Fossen 2021, Eq. 3.47}),$$

with velocities $\nu = [u, v, r]^T$ and kinematics $\dot{\eta} = R(\psi)\nu$, simulating the vessel dynamics and propagating its states over time can be achieved by following the steps in the pseudocode below

```
% Initialize x <- x0, t <- 0
for k = 0 ... N-1:
    % 1) Read or compute inputs from tau_RB at time t
    u_in = [delta_c; n_c] % any relevant inputs
    tau_RB = B * u_in % external forces and moments in BODY

    % 2) Compute rigid body dynamics from current state and inputs
    C = CRB(nu) % from current nu
    nu_dot = inv(MRB) * ( tau_RB - C * nu ) % solve for accelerations

    % 3) Kinematics - Map BODY velocities to NE heading rates
    eta_dot = R_zyx(0,0,psi) * nu

    % 4) Integrate states in time, e.g. using RK4
    x <- RK4( f(x, u_in), x, h )

    t <- t + h
end
```

Bibliography

- [1] Thor I. Fossen. *Handbook of Marine Craft Hydrodynamics and Motion Control*. 2nd ed. Chichester, UK: John Wiley & Sons, 2021. ISBN: 9781119575054.

Appendix

A Code for task 4a

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Add added mass here
MA = [-Xudot 0 0
      0 -Yvdot -Yrdot
      0 -Nvdot -Nrdot];
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% rigid-body mass matrix
MRB = [ m 0 0
        0 m m*xg
        0 m*xg Iz ];

Minv = invQR(MRB + MA);

% input matrix
t_thr = 0.05; % thrust deduction number
X_delta2 = 0; % rudder coefficients (Section 9.5)
Y_delta = 0;
N_delta = -1e7;
Bi = [ (1-t_thr) X_delta2
       0 Y_delta
       0 N_delta ];

% state-dependent time-varying matrices
CRB = m * nu(3) * [ 0 -1 -xg
                    1 0 0
                    xg 0 0 ];

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Add Coriolis due to added mass here
CA = m2c(MA, nu); % From MSS toolbox
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

R = Rzyx(0,0,eta(3));

% thrust
thr = rho * Dia^4 * KT * abs(n) * n; % thrust command (N)

% ship dynamics
u = [ thr delta ]';
tau = Bi * u;
nu_dot = Minv * (tau - (CRB + CA) * nu);
eta_dot = R * nu;
```

B Code for task 4b

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Add linear damping here
T1 = 20; T2=20; T6 = 10;
Xu = (m - Xudot)/T1;
Yv = (m - Yvdot)/T2;
Nr = (Iz - Nrddot)/T6;
Dlin = diag([-Xu, -Yv, -Nr]);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Add nonlinear damping here
% --- Parameters for nonlinear surge drag ---
k = 0.1; % form factor
CR = 0.0; % residual resistance coeff
eps = 1e-3; % small epsilon for log10 robustness
nu_k = 1.19e-6; % kinematic viscosity [m^2/s] (sea water)
Vc = 0.5; % Ocean current velocity [m/s]
beta_c = pi/2; % Ocean current angle [rad]
u_c = Vc*cos(beta_c - eta(3)); % Ocean current velocity relative to ship in surge
    % [m/s]
v_c = Vc*sin(beta_c - eta(3)); % Ocean current velocity relative to ship in sway
    % [m/s]

% --- Relative velocities ---
nu_r = nu - 0*[u_c; v_c; 0]; % Calculate relative velocities
u_r = nu_r(1); v_r = nu_r(2); r = nu_r(3);

% --- ITTC-57 friction line ---
Re = abs(u_r)*L /nu_k;
Cf = 0.075 / ( (log10(Re + eps) - 2)^2 );
Ct = (1 + k)*Cf + CR;

% --- Simple wetted surface approximation ---
Sw = 2*(L*T + L*B + B*T); % [m^2]
Xnl = -0.5 * rho * Sw * Ct * abs(u_r) * u_r; % nonlinear surge force [N]

% --- Cross-flow drag (strip theory) ---
Ycf = 0;
Ncf = 0;
dx = L/10; % 10 strips
Cd_2D = Hoerner(B, T); % MSS toolbox 2D Cd

for xL = -L/2 : dx : L/2
    Ucf = abs(v_r + xL*r) * (v_r + xL*r); % |v_loc| v_loc
    Ycf = Ycf - 0.5 * rho * T * Cd_2D * Ucf * dx; % sway force
    Ncf = Ncf - 0.5 * rho * T * Cd_2D * xL * Ucf * dx; % yaw moment
end

% --- Total hydrodynamic damping force---
tau_damp = (Dlin*[u_r; v_r; r] + [Xnl; Ycf; Ncf]);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

R = Rzyx(0,0,eta(3));

% thrust
thr = rho * Dia^4 * KT * abs(n) * n; % thrust command (N)
```

```
% ship dynamics
u = [ thr delta ]';
tau = Bi * u;
nu_dot = Minv * (tau - (CRB + CA) * nu + tau_damp);
eta_dot = R * nu;
```

C Code for task 4c

```
%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% USER INPUTS
%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
clc; clear; close all;
T_final = 1000;           % Final simulation time (s)
h = 0.1;                  % Sampling time (s)
control_law = 0;          % Choosing the control law

psi_ref = 10 * pi/180;    % desired yaw angle (rad)
u_ref = 7;                % desired surge speed (m/s)

% initial states
eta = [0 0 0]';
nu = [0 0 0]';
delta = 0;
n = 0;
x = [nu' eta' delta n]';

%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% MAIN LOOP
%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
t = 0:h:T_final;          % Time vector
nTimeSteps = length(t);   % Number of time steps

simdata = zeros(nTimeSteps, 13); % Pre-allocate matrix for efficiency

for i = 1:nTimeSteps

    % reference models
    psi_d_deg = psi_ref;
    r_d_deg = 0;
    u_d = u_ref;

    if control_law == 1
        % --- time-varying step response commands ---
        t_now = t(i);
        delta_c = 0.1; % [rad]
        n_c = 10; % [rps]

        % Step 1: rudder step (100{300 s)
        if t_now >= 300 && t_now < 800
            delta_c = 0.5; % 5 deg rudder step
            n_c = 10; % propeller off
        end
    elseif control_law == 2

        % --- time-varying step response commands ---
        t_now = t(i);
        delta_c = 0.1; % [rad]
        n_c = 10; % [rps]

        % Step 2: propeller step (600{800 s)
        if t_now >= 300 && t_now < 800
            delta_c = 0.1; % rudder neutral
            n_c = 20; % propeller speed step (rps)
        end
    end
end
```

```
else
    % control law
    delta_c = 0.1;           % rudder angle command (rad)
    n_c = 10;                % propeller speed (rps)
end
```